要点梳理

- **NLP**
  - **任务**：分类、聚类（分组）、回归、生成
    > LLM 实现：全部变成生成
  - **模型**：RNN、LSTM、Transformer、BERT
- **Transformer**

  datawhalechina/learn-nlp-with-transformers
  - **输入**：文字 $\xrightarrow{Tokenization}$ Token + 编号 $\xrightarrow{Embedding}$ 向量（+ 位置编码）
  - **输出**：向量序列 + 概率分布 ⟶ 选概率最高的词
  - **Encoder**：x1~xn $\xrightarrow{Self-Attention}$ z1~zn $\xrightarrow{Add+Normalize}$ $\xrightarrow{FFN}$ r1~rn $\xrightarrow{Add+Normalize}$ 输出向量
    - **Self-Attention**：$Attention(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V = Z$
    - **FFN（Feed-Forward Network）**：对每个输入做非线性变换
    - **Add（残差连接）**：Output = Attention(x) + x
    - **Normalize（层标准化）**：映射到稳定区间
  - **Decoder**：
    - 解码器输出的 r1~rn $\xrightarrow{*Wk\ or\ *Wv}$ K V 矩阵
    - 当前生成的序列 $\xrightarrow{Masked自注意力}$ 当前Qt
    - K V 矩阵 + Qt $\xrightarrow{源注意力}$ 输出向量 $\xrightarrow{FFN}$ 输出向量 $\xrightarrow{Linear}$ $\xrightarrow{Softmax}$ 输出单词
      - **Linear（线性层）**：向量映射到单词表，每个单词有个分数
      - **Softmax**：归一化（概率分布）
- **LLM**
  - **训练**：预训练（语言建模） ⟶ 微调（Finetuning/SFT） ⟶ RLHF（对齐价值观）
    - 微调：全参微调（Full FT）、参数高效微调（PEFT）、LoRA（低秩自适应）
    - 评测：LM Evaluation Harness（lm-eval）
  - **优化**：输入数据 ⟶ Forward（猜测结果） ⟶ 计算损失（损失函数） ⟶ Backward（优化器，修改权重） ⟶ 循环
    - 损失函数：交叉熵函数（Cross-Entropy Loss） $Loss = -\sum_{i=1}^{n} y_i \log(p_i)$
  - **RAG（检索增强生成）**：

    > 解决知识时效性、幻觉、私有数据安全
    - 数据 ⟶ 分块 $\xrightarrow{EmbeddingModel}$ 向量 ⟶ 存入向量数据库
    - 用户提问 $\xrightarrow{EmbeddingModel}$ 向量 ⟶ 检索向量数据库
    - 检索到的信息 +用户问题 ⟶ 合成 Prompt ⟶ LLM 生成回答

- 提示词工程：
  - 一般准则：设定角色、背景数据隔离、行为准则、输出格式、给出示例
  - CoT：Let’s think step by step
  - Few-shot Learning：给出一些 QA 对
- Agent
  - 结构：$LLM \overset{FunctionCalling}{\rightleftharpoons} Agent \overset{MCP}{\rightleftharpoons}$ 工具程序集
    - **LLM**：决策层，负责思考用户需求需要用到什么工具
    - **Function Calling**：让 LLM 结构化格式输出指令给 Agent
    - **Agent**：执行调度层，接收 LLM 指令并执行，接收 LLM 回答并返回用户
    - **MCP（模型上下文协议）**：让 Agent 能够调用各种工具
    - **Skill**：`skill.md` 说明文档
  - Agent 开发三层次
    - **底层（Runtime）**：Agent 可靠运行的底层基础设施 —— **LangGraph**

      > 持久化执行、人机协作、流式输出、状态管理
    - **中间层（Framework）**：Agent 怎么快速开发 —— **LangChain**

      > 模型抽象、Agent 循环、中间件、工具接口
    - **上层（Harness）**：Agent 怎么开箱即用 —— **DeepAgents**

      > 任务规划、子 Agent 委派、文件系统、长期记忆
  - LangChain
    - **代码实现**

      > 1. 定义组件：LLM、Prompt、Memory（可选）
      > 2. 构建外部能力（可选）：RAG、Tools、MCP
      > 3. 定义输出格式（可选）：OutputParser
      > 4. 构建执行逻辑：Chain（固定流程） 或者 Agent（动态决策）
      > 5. 得到可调用对象
    - **执行流程**

      > 1. 用户输入 → LangChain 构造完整 Prompt → LLM：系统指令、工具说明、Memory（可选）
      > 2. LLM 决策（语义理解，决定是否调用工具、是否 RAG 等） → 输出结构化指令给 LangChain
      > 3. LangChain 解析指令并执行：包括调用工具（可用MCP）、RAG 等
      > 4. LangChain 接收执行结果（MCP回传） → Prompt → LLM
      > 5. LLM 生成回答（可选 OutputParser 结构化） → LangChain → 用户
  - DeepAgents
    - `create_deep_agent` **参数**
      - `model`：使用的 LLM
      - `system_prompt`：定义 AI 的角色身份
      - `tools`：工具（通用）
      - `skills`：目录路径，里面是 md 文档，对 Tool 的逻辑指导（具体）

- `memory`：纯文本参考资料，运行期注入
- `backend`：提供 Agent 的执行环境（虚拟文件系统）

  > StateBackend、StoreBackend、FilesystemBackend、LocalShellBackend、CompositeBackend、Sandbox（Modal / Runloop / Daytona / LangSmith）

- `middleware`：在节点的前后执行
- `subagents`：隔离工作
- `response_format`：结构化输出

- 记忆
  - **运行流程**：Graph（Node+Edge）与 GraphState（对话历史）
  - **会话记忆**：定义 Checkpointer 存储 State 数据（根据 thread_id）
  - **跨会话记忆**：定义 Store 对象，连接 StoreBackend 或CompositeBackend（根据 user_id）